

SE LAB 3

Problem Statement #36:

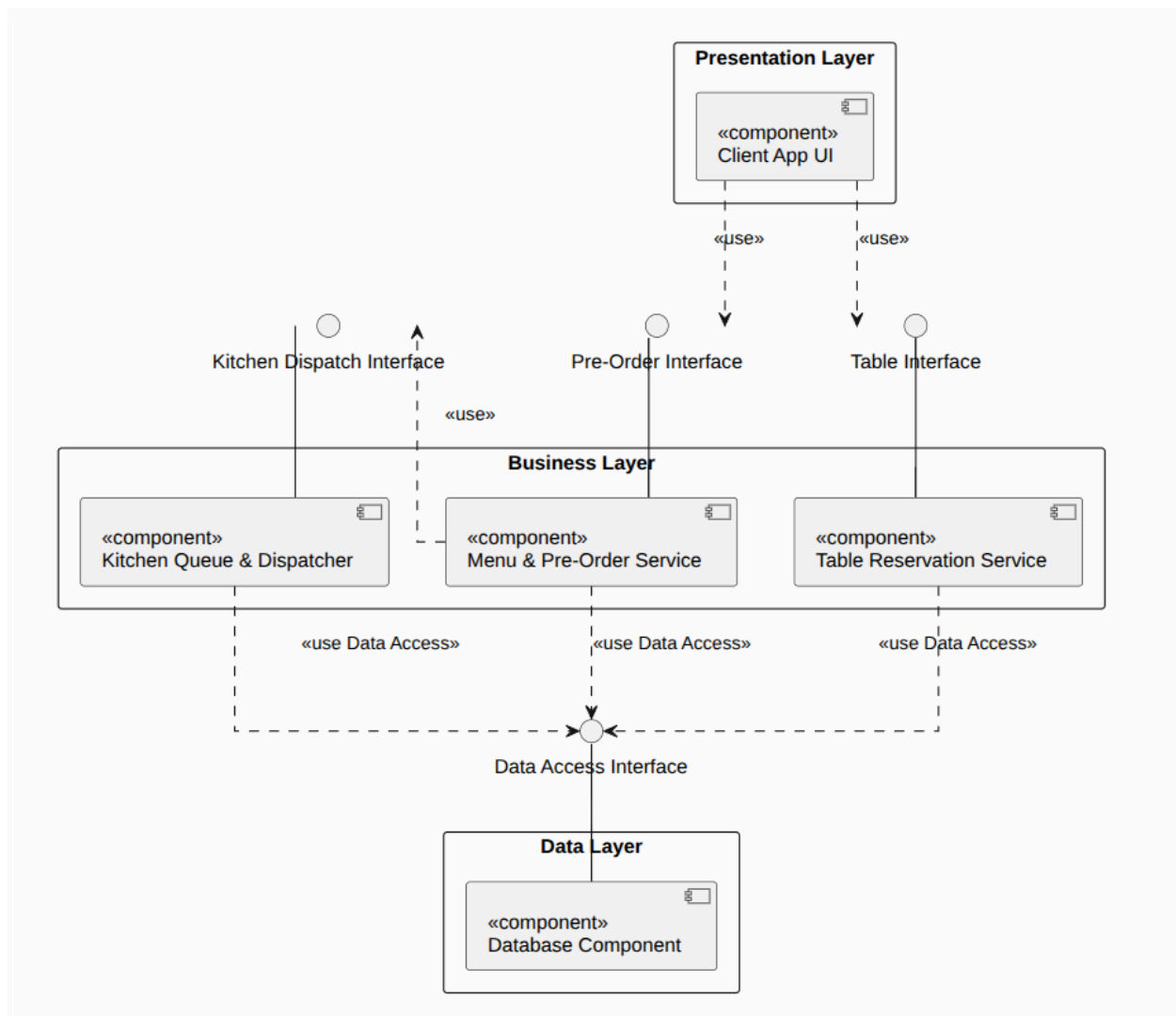
Restaurant Table Booking & Pre-Ordering App

GitHub: <https://github.com/ANDYGAMER4321/SE>

Ayushman Dey

PES2UG24CS102

Component Diagram:



Written Justification:

Architecture Selection:

I chose Layered Architecture for the Restaurant Table Booking & Pre-Ordering App.

Reason 1:

Natural Separation of Workflow and Lifecycle Concerns

The system encompasses distinct operational lifecycles: interactive floor plan table selection, food pre-ordering, and kitchen prep dispatching. A layered architecture isolates presentation rendering (such as displaying the 2D floor grid on diner apps and floor terminals) from business logic (preventing double-booking and tracking preparation timelines). This separation allows validation logic or dining-window rules to change without impacting client interfaces or data storage layers.

Reason 2:

High Cohesion with Minimal Operational Complexity

This platform targets a single-restaurant ecosystem of diners and manager floor terminals. A microservices architecture would introduce unnecessary overhead, including network latency, service discovery, and complex distributed transaction management across reservations and carts. A 3-tier layered monolith provides clean modular boundaries and simple deployment while keeping transactional operations fast and reliable.

Security Advantage:

Isolating the Database Component within a dedicated Data Layer prevents client-facing components from querying the database directly. Role-Based Access Control (RBAC) is enforced centrally in the Business Layer: diner sessions cannot modify table availability flags or access manager floor controls. All requests must pass through validated business interfaces before any read or write transaction reaches the database.

Performance Benefit:

To meet the requirement of synchronizing floor table status changes (Reserved, Seated, Billed) across all manager terminals in under 500 ms (NFR-001), the Table Reservation Service operates as an independent business component. It updates state records directly through optimized Data Access interfaces without waiting on slower pre-order processing or kitchen preparation time calculations.

